You said:

read the transcript below for 9/30 lecture: people is that we have TA office hours today. Uh so the TAs told us that uh 0:14 last week the attendance at the office hours was extremely sparse to the point of some of them having zero. So if we're 0:21 not going to you know if people want TA office hours, you know, you should go to TA office hours. If we think see things 0:27 being attended at level zero, then we'll like divert those resources elsewhere. Um, so we we've heard people's requests 0:34 of uh possibly trying to have uh an office hour on Friday. We're looking into it. We're seeing if we can possibly 0:41 do it, but you should come to office hours today if you have uh questions. Remember the solutions are in scope from 0:47 last week. The homework that is currently happening this week is in scope. So you know take advantage of it 0:53 to understand. Also want to remind people about discussion. Discussion tomorrow will go 0:59 through and work you through lots of the basics of uh CNN's. It's good to solidify your information. There's also 1:05 be also going to be a com you're not going to let it sit idle, let those tensor cores sit idle. You want to use them. How do you 1:17:08 use them? You put more images in as your batch. 1:17:14 Okay? So you have a batch. So you have lots of things that one can 1:17:20 average over. So one of the first kinds of normalization layers in the modern era 1:17:27 of normalization layers was a normalization layer called batchnorm. 1:17:36 What happened here? 1:17:44 So what was batchorm? Bashnorm said we're going to do the same thing that we 1:17:51 do for input standardization except we're going to do it at an intermediate layer. So for input 1:17:58 standardization what do you do? You take all of your inputs and for every particular position in your input for a 1:18:04 vector input you take the mean subtract it off and you set set it so that across 1:18:11 all your inputs the standard deviation is one. Okay. So what you do is you do averaging 1:18:19 this way across the batch. 1:18:27 But to get more averaging and to respect the idea of what what your spatial 1:18:33 filters are like, you also average across space 1:18:38 in a convent. 1:18:51 Okay, so bashnorm does that. So this averages 1:19:01 over space and batch 1:19:08 not over channels. The logic behind averaging over space is to say that you know you might have a cat somewhere in 1:19:15 the image. You want to make sure that different positions of the image are all represented. Okay, so you average over 1:19:21 all the entire image. So this is uh what batch norm does and 1:19:28 let me just write out uh the expressions. So 1:19:35 let curly be you know 1:19:42 the what is 1:19:48 averaged over. Okay. So for the different norms it'll be different. For bashn norm it's batch and space. 1:19:56 You first compute a mean 1:20:11 of whatever the input is then you compute 1:20:18 a scaling factor. So again it as before 1:20:25 one over Sorry. The size of Okay, that's very messy. 1:20:38 Sum over everything being averaged 1:20:45 hi minus the mean squared. So this is the standard deviation and 1:20:50 this is the mean. Yeah. using all the locality information. 1:20:56 Okay, hold on a minute. All I'm trying to do is to understand how to scale. I will keep all the locality information. 1:21:02 I'm just going to rescale everything. So, I'll just write the next thing out. What I could do is to say h out, 1:21:11 okay, as a vector is going to be. 1:21:20 So let's look at H out in the in a particular position I comma J H out in 1:21:26 the I comma J position to keep the locality clear. One thing you could do is just to take H 1:21:34 input. So okay sorry I'll write input in okay H 1:21:40 in the I comma J position except I'm going to subtract the mean 1:21:46 and I will divide by this standard deviation regularized by an epsilon. 1:21:51 Okay, this would standardize it to have average zero over the averaging domain 1:21:59 and variance one. Everyone with me with this? But in practice, we also add 1:22:05 learnable parameters. So, sorry, I'm out of time. 1:22:20 Okay. So, I'll pick up here next time and continue with the other other normalization layers next time. 1:22:32 Did you think of batch norm as kind of like getting rid of an implicit bias across batches because then like batches 1:22:40 in one channel where like maybe the values are higher those would have minor gradients of the loss. So you're kind of 1:22:47 like I mean it's easier to think of batch norm as just being you are doing input 1:22:53 standardization for an intermediate layer and you're doing it on the entire size of input you have which just happens to can you explain why it scales by square root when hes talking about backprop and when all dl/dw is the same

ChatGPT said:

Sure — here is the clean explanation of **why the scale of the gradient grows like √N** when the individual ∂L/∂w contributions are uncorrelated (or random-ish), exactly as the professor explained in the transcript.

✓ Context

We are in the part of lecture where the professor explains this situation:

- A convolution filter weight **w** is shared in many spatial positions.
- Therefore **∂L/∂w** is the *sum* of the gradients from each spatial location.

Mathematically:

$$\frac{\partial L}{\partial w} = \sum_{i=1}^{N} \frac{\partial g_i}{\partial w} \quad \frac{\partial L}{\partial = i=1} \sum N g i$$

where each gigi is the gradient coming from one spatial location.

He then asks:

> *If these gigi are "randomish" or uncorrelated, how does the magnitude of the sum grow with N?*

⭐ Why does it scale like √N?

Case 1 — All gradients point in exactly the same direction

(i.e., all gigi are equal)

g1=g2=⋯=gN=gg1=g2=⋯=gN=g

Then:

∂L∂w=Ng∂w∂L=Ng

Magnitude grows linearly with N.

This is the "all correlated" case (e.g., 100 terms → scale ×100).

In this exchange, I really liked how the AI broke the √N scaling into two separate cases: perfectly correlated gradients (scale like N) versus uncorrelated "random-ish" gradients (scale like √N). That made the professor's point about weight sharing much clearer to me than in the live lecture, where it went by pretty fast. The random-walk/central-limit-theorem analogy also helped me build some intuition for why lots of small, noisy contributions don't simply cancel out. That said, the AI leaned a bit into extra applications like FP8/FP16 overflow and "electrical noise accumulation," which were not directly mentioned in lecture and might be a bit distracting if you're just trying to connect to Sahai's slide. Overall, though, for this concept it stayed faithful to the lecture: the use of variance and standard deviation to derive √N scaling is mathematically correct, and the explanation about why we consider the magnitude of the *sum* rather than the average is consistent with what showed up on the board.

Case 2 — Gradients are *uncorrelated* (random signs, random small numbers)

Assume:

- E[gi]=0E[gi]=0
- Var(gi)=σ2Var(gi)=σ2
- The gigi's are independent

Then:

S=∑i=1NgiS=i=1∑Ngi

has:

Var(S)=Nσ2Var(S)=Nσ2

so:

Std(S)=NσStd(S)=Nσ

> **The scale (expected magnitude) grows like √N.**

This is exactly the same math as:

- random walk distance
- central limit theorem scaling
- adding independent noise vectors
- electrical noise accumulation
- diffusion scaling

Why the professor emphasized this

Because in CNNs:

- A single filter may be applied at **H × W ≈ 250 × 250 = 62,500 spatial locations**.
- So ∂L/∂w is the sum of ~60k terms.

If gradients are random-ish (not strongly correlated), then:

‖∂L/∂w‖∼HW‖∂L/∂w‖∼HW

Example with a 256×256 image:

2562=65,536≈2562562=65,536≈256

so the gradient magnitude is about ×256 bigger than the per-location gradient.

That's why:

- **gradient explosion** is a real problem
- normalization layers act like **speed bumps**
- FP8/FP16 precision can overflow
- Xavier/He initialization matters
- RMSNorm/LayerNorm help stabilize scaling

Why your confusion happened

You thought:

> If all ∂L/∂w contributions are the same, we should average, not sum.

But the professor separated **two cases**:

✓ Case A — contributions truly identical

→ gradient = N×g.

✓ Case B — contributions different/random

→ magnitude ≈ √N.

Most CNN layers fall somewhere in between — not fully correlated, not fully random.

Intuition (best summary)

Adding N random bumps → grows like √N.
Adding N identical bumps → grows like N.

Max pooling → routes only one gradient
Conv → sums many gradients
Normalization layers → prevent √N or N growth from exploding training

---

If you want, I can also explain:

- How RMSNorm prevents this growth
- Why MUP scaling assumes RMS=1
- How this interacts with weight sharing
- Why summing is mathematically required (not averaging)

Just tell me!

You said:

what was his point about backprop and htat part of the lecture. is he saying to average or not to average

ChatGPT said:

Here's **exactly** what his point was in that part of the lecture, in the simplest possible terms:

✅ His point: In backprop for shared weights, you MUST SUM gradients — NOT average them.

But he then explained:

- **Why the sum mathematically appears** (chain rule)
- **Why the scale of the sum behaves like √N** when the gradients are random
- **Why this scaling can cause problems (gradient explosion)**
- **Why normalization layers exist partially to control this scaling**

So the answer is:

⭐ **He is NOT saying we should average.

He is explaining why summing causes big gradients, and why normalization helps.**

This is a good example of the AI functioning as a "translator" for lecture language. In class, the professor casually contrasted "adding" vs "averaging" the gradients, and it was easy to walk away thinking maybe we *should* average to prevent explosions. Here, the AI explicitly says "he is NOT saying we should average" and ties that back to the chain rule formula for shared weights. That direct yes/no framing helped clear up my confusion. The rest of the paragraph, where it explains that the √N story is descriptive (what happens) rather than prescriptive (what we should implement), is also aligned with the lecture. If anything, the answer might be longer than necessary; it restates "we must sum, not average" several times. But as a pre- or post-lecture reading, that redundancy is actually useful: it hammers home that averaging would literally be changing the math of backprop, not just a harmless implementation detail.

🧠 Why we SUM (not average)

When a weight ww appears in many places in the computational graph (weight sharing):

$$\frac{\partial L}{\partial w} = \sum_i \frac{\partial L}{\partial w_i}\frac{\partial w_i}{\partial w} \frac{\partial L}{\partial w} = \sum_i \frac{\partial L}{\partial w_i}$$

This comes directly from the **chain rule**.

He even stated:

> "Why are we adding instead of averaging? Because that's what the chain rule dictates."

So where does √N come in?

He was **not** saying "use √N" or "average by √N".

He was explaining what **happens** (not what we should do):

- If the gradients from different positions are *uncorrelated*, then the **magnitude** of the sum grows like √N.
- If they're perfectly aligned, magnitude grows like N.

He used this to explain why:

✓ CNN gradients can get huge

✓ FP16 / FP8 can overflow

✓ RMSNorm and BatchNorm help stabilize training

✓ Weight sharing isn't free; it amplifies gradients

But **he never said to average**.

The real message of that whole section

1. **Backprop through shared weights = sum of all gradient contributions.

Never average. That would be wrong mathematically.**

2. This sum can grow large.

- If random → √N
- If correlated → N

3. Normalization layers help keep gradients stable because this natural scaling can cause overflow or instability during deep training.

4. This is tied to the earlier MUP discussion:

- MUP assumes RMS norms stay order 1
- But repeated updates make RMS norms drift
- So normalization enforces stability when MUP's assumptions break over many updates

---

TL;DR (super clear)

Is he saying we should average?
✗ NO.

Is he saying we must sum?
✓ YES — chain rule requires it.

Why did he talk about √N?
✓ To explain how the magnitude of the sum behaves and why training can become unstable.

Why is this important?
✓ To motivate RMSNorm / BatchNorm — they prevent gradient growth caused by these sums.

If you want, I can give a short 5-line summary of that entire concept, or explain how this connects to MUP scaling and Xavier initialization.

You said:

can you explain the one quarter hittign this thing he was saying in the pooling portion

ChatGPT said:

Here the AI basically re-derives exactly what Sahai did on the board for 2×2 mean pooling, but in slower motion. I like that it explicitly writes out $\partial y/\partial a = 1/4$ etc. and then uses the chain rule to show that each of a, b, c, and d gets g/4. During lecture, this went by as a quick verbal "one quarter each," and I didn't fully internalize that it's just the derivative of the average. This section doesn't seem to hallucinate anything; it sticks to the simple 2×2 case and doesn't invent extra details outside the course. The only minor critique is that it doesn't connect back to the bigger picture that Sahai was pushing: in mean pooling, gradients are spread out across inputs, while in max pooling they are sparse and only go to the argmax. Still, for understanding this local computation, this answer works as a very effective step-by-step supplement.

Yes — here is **exactly** what he meant by “one quarter of the gradient hits each thing” in the **mean pooling** part of the lecture.

This is simple once you see it.

Context: Mean Pooling (2×2 pool example)

Suppose the input to the pooling window is:

```
a b c d
```

Mean pooling computes:

$y = \frac{1}{4}(a+b+c+d)$

Backpropagate a gradient through this

Let the gradient from the next layer be:

$\frac{\partial L}{\partial y} = g$

We now need the gradient **for each input element** a, b, c, d.

Compute:

$\frac{\partial y}{\partial a} = \frac{1}{4}, \frac{\partial y}{\partial b} = \frac{1}{4}, \frac{\partial y}{\partial c} = \frac{1}{4}, \frac{\partial y}{\partial d} = \frac{1}{4}$

$\frac{\partial a}{\partial y} = 4, \frac{\partial b}{\partial y} = 4, \frac{\partial c}{\partial y} = 4, \frac{\partial d}{\partial y} = 4$

Chain rule:

$\frac{\partial L}{\partial a} = g \cdot \frac{1}{4} \frac{\partial a}{\partial L} = g \cdot 4 \frac{\partial L}{\partial b} = g \cdot \frac{1}{4} \frac{\partial b}{\partial L} = g \cdot 4 \frac{\partial L}{\partial c} = g \cdot \frac{1}{4} \frac{\partial c}{\partial L} = g \cdot 4 \frac{\partial L}{\partial d} = g \cdot \frac{1}{4} \frac{\partial d}{\partial L} = g \cdot 4$

⭐ Therefore: each of the 4 inputs gets exactly 1/4 of the gradient.

This is what he meant by:

> “Effectively what happens is whatever gradient was hitting this,
> one quarter of that will hit each of these positions.”

Intuition

Mean pooling averages the pixels → so during backprop, the gradient also gets “averaged” back across the 4 contributing pixels.

✓ Summary

Forward-pass of mean pool:

$y = \frac{1}{4}(a+b+c+d)$

Backward-pass:

$\nabla a = \frac{1}{4} \nabla y \quad \nabla b = \frac{1}{4} \nabla y \quad \nabla c = \frac{1}{4} \nabla y \quad \nabla d = \frac{1}{4} \nabla y$

If you want, I can explain the max pooling backprop part too—why the gradient only goes to the argmax and **0** to the rest.

You said:

how did you get dl dy = g? what is g

ChatGPT said:

Great — let me explain that clearly.
“g” is just a placeholder name for the gradient coming **from the next layer** during backprop.

Let’s go step-by-step so you see exactly where it comes from.

✅ **1. Backprop always works *from the output layer backward***

You always receive a gradient from the next layer.**

If you have:

```
input → pooling → next_layer → loss
```

Then during backprop:

- The **next layer** computes its gradient w.r.t. its input.
- That gradient becomes the **incoming gradient** to the pooling layer.

We usually name this incoming gradient:

∂L∂y∂y∂L

Where:

- yy = output of the pooling operation
- LL = loss

Instead of writing that entire symbol every time, lecturers often give it a short name:

∂L∂y=g∂y∂L=g

So:

“g” = the gradient coming into the pooling layer from above.

It’s just a shorthand.

This is one of the spots where the AI behaves kind of like a TA (shoutout Sultan!): it doesn’t just answer the symbol-pushing question, it backs up and explains the pattern of backprop. The lecture assumes everyone is comfortable with “the gradient from the next layer,” but in practice that “mystery arrow” is often the confusing part. By naming ∂L/∂y as “g” and emphasizing that this is just the incoming gradient from the layer above, the AI makes the notation in Sahai’s diagrams much easier to parse. I also appreciate that it stresses we don’t care *why* g has a certain value at this point; we just treat it as the input to our local chain rule. That’s a helpful mental model. The explanation is a bit verbose and could be shorter for someone already fluent with autodiff, but for a pre-lecture warm-up I think this level of hand-holding is exactly what a lot of 182 students need.

🔢 2. Why can we rename it to “g”?

Because we don’t care what caused it — only that it exists.

Backprop is recursive:

- The next layer already computed **its** derivative.
- That number is handed *back* into the pooling layer.

Example:
If the next layer says “the loss changes by 0.7 for a small change in y”, then:

∂L∂y=0.7∂y∂L=0.7

We call that number “g”.

---

3. Then we continue backprop through pooling

For 2×2 mean pooling:

y=14(a+b+c+d)y=41(a+b+c+d)

Chain rule:

∂L∂a=∂L∂y⋅∂y∂a=g⋅14∂a∂L=∂y∂L⋅∂a∂y=g⋅41

Same for b, c, d.

That's where the **1/4 hitting each input** comes from.

TL;DR — What is g?

- $g = \partial L \partial y g = \partial y \partial L$
- It is the gradient that comes **into** the pooling layer from the next layer.
- It's called "g" just to simplify notation.

If you want, I can show you the *full chain rule expression* without abbreviations so you can see how the g enters mathematically.

You said:

in the pool like abcd, what are these numbers exactly and what do they represent

ChatGPT said:

Great — let's make this **super concrete** because this is one of those things that suddenly "clicks" once you know exactly what *a, b, c, d* are.

✅ What are a, b, c, d in the pooling example?

They are simply the **four pixel values** (or feature values) in the **2×2 pooling window** at some channel.

Like this:

```
a b c d
```

They are **not weights**, **not activations from a dense layer**, and **not loss values**.

They are literally:

✓ the four numbers from the feature map (output of the previous layer)

✓ inside the pooling region

✓ at a specific channel

Example (Concrete Values)

Suppose the previous convolution layer produced this feature map slice:

```
13 7 5 9 4 2 8 3 11 6 1 0 10 3 2 4
```

Now a 2×2 mean pool with stride 2 will process it by windows:

Window 1:

```
a=13 b=7 c=4 d=2
```

Window 2:

```
a=5 b=9 c=8 d=3
```

In this segment, the AI does a nice job of grounding the math in a picture: it explicitly says that a, b, c, d are just the four feature values in a 2×2 window of the feature map, not weights or anything fancier. I found the example with actual numbers in a 4×4 grid especially helpful because it matches the kinds of visualizations Sahai showed earlier when explaining convolutions. Conceptually, this aligns well with the lecture's story: convolutions produce feature maps where each channel is detecting patterns, and pooling then aggregates those local responses. There is a little bit of extra narrative flavor ("is there a vertical edge here?") that goes beyond what was literally said in class, but it's still consistent with standard CNN intuition. This is an example where the AI feels like a good "pre-lecture reading": if I had read this before the pooling part of the lecture, I would have been much less confused by the notation when Sahai wrote down a, b, c, d on the board.

Window 3:

```
a=11 b=6 c=10 d=3
```

Window 4:

```
a=1 b=0 c=2 d=4
```

Each window gives **one pooled output**.

🧠 What these numbers represent conceptually

These values are:

the features that the convolutional filters detected

at that position of the input image.

Examples of what a conv feature might represent:

- “Is there a vertical edge here?”
- “Is there a cat’s ear shape here?”
- “Is there a texture pattern here?”
- “Is this area bright/dark/colorful?”

Pooling takes these local activations and:

✓ summarizes them

✓ makes the representation more invariant

✓ compresses the spatial resolution

How they interact with pooling

Mean Pooling

y=a+b+c+d4y=4a+b+c+d

Pooling says:

> “Instead of remembering all 4 numbers, I’ll keep their average.”

Then backprop:

If gradient coming from next layer = g

Each input receives:

∂L∂a=14g∂a∂L=41g

Same for b, c, d.

⭐ Key idea: a, b, c, d = the four inputs inside the pooling window — nothing else.

They are the outputs of the previous layer, taken from a small spatial neighborhood.

⚡ Quick visual summary

Feature map slice → pooling window:

```
Previous layer feature map: [... a b ...] [... c d ...] [... ... ... ...] Mean Pooling → output = (a+b+c+d)/4 Max Pooling → output = max(a,b,c,d)
```

Backprop:

- Mean → gradient splits evenly (¼ each)
- Max → gradient routed to the max element only

If you want, I can also explain:

- Why max pooling sends **all gradient only to the argmax**
- Why people often use max pooling instead of mean pooling
- Or how pooling differs from strided conv (they are very related)

Just tell me!

This answer is useful because it distinguishes clearly between *activations* and *weights*, which is something the lecture implicitly assumes but doesn't always spell out step-by-step. In class, it's easy to get the impression that we are "updating a, b, c, d" directly, since those are the numbers being drawn on the slide. The AI corrects that by repeatedly emphasizing that only the *weights* (in the convolution layer before pooling) are learnable parameters. I like the concrete mini-timeline: backprop routes gradients through pooling → gradients reach conv weights → optimizer updates weights → next forward pass produces new a, b, c, d. This matches the conceptual flow described in lecture and doesn't introduce any obvious hallucinations. If anything, the explanation is more detailed than necessary, but I think that's an advantage for self-study: it would let a student pause and trace how the whole training loop works, instead of only seeing a static snapshot.

You said:

right so they get changed after next pass right?

ChatGPT said:

Yes — **exactly.**
Those numbers (*a, b, c, d*) get updated **after backprop**, but *not directly*.
Let me explain clearly how they change.

✅ **Key Idea: a, b, c, d do NOT change themselves —

the WEIGHTS before them get updated, which PRODUCES new a,b,c,d next forward pass.**

This is the chain:

Forward pass:

Conv layer produces values → they appear as **a, b, c, d** inside the pooling window.

Backward pass:

Pooling layer computes gradients:

- Mean pool: each gets ¼ of the gradient
- Max pool: only the max gets the gradient

These gradients do **not directly modify** a,b,c,d.

They flow *backward* to the previous conv layer.

Optimizer step:

The gradients flow into the **weights of the previous conv layer**.

The optimizer (SGD, Adam, Muon, etc.) updates the weights.

Next forward pass:

New weights → recompute the feature map → the values at those positions change.

So:

⭐ Yes, a,b,c,d change every iteration — but only because the upstream conv layer weights were updated.

🧮 Let's make it concrete

Imagine on some iteration:

```
a=13 b=7 c=4 d=2
```

Pooling backprop produces gradients for these 4 inputs:

Mean pool → each gets g/4
Max pool → only one gets g, rest get 0

These gradients are used to update the conv filters.

After updating the conv filters, the next forward pass produces *new* values:

Example:

```
a'=11 b'=8 c'=3 d'=1
```

Because the **weights changed**, the generated feature map changed.

**So you are 100% correct:

a, b, c, d change on the next pass.
They change because the CONV weights change.
Pooling is just routing gradients — not storing learnable parameters.**

Pooling itself has **no learnable parameters** (unless you use things like learned pooling, but standard max/mean pool have none).

Summary

✓ a,b,c,d are the input activations to the pooling window
✓ Backprop sends gradients from pooling → previous layer
✓ Optimizer updates the previous layer's weights
✓ Next forward pass creates new a,b,c,d
✓ Pooling does NOT have parameters, it only routes gradients

If you want, I can also explain:

- How gradients flow from pooling → conv → earlier layers
- Why max-pool blocks gradient to 3 out of 4 inputs
- How this creates sparsity in gradient flow

Just say the word.

You said:

does max pooling increase channels or does liek a convolution increase channels i am on the alexnet part of the lecture

ChatGPT said:

Great question — and**this is exactly the distinction the professor wanted you to understand when he showed AlexNet.**

Let's answer it cleanly:

⭐ Short Answer

❌ Max pooling does *NOT* increase channels.

✓ Convolution layers are the ONLY thing that increase (or decrease) channels.

Why max pooling does NOT change channels

Max pooling operates **independently on each channel**.

For every input channel, it produces exactly **one output channel**.

Example:

If the input to max pooling is:

- 256 channels
- each channel is a 13×13 spatial map

Then after 2×2 max pool:

- still **256 channels**
- but each is now 6×6 (spatial downsample)

So pooling changes **height & width**, but **never the number of channels**.

Why convolution DOES change channels

A convolution with **C_out** filters produces **C_out output channels**.

Example:

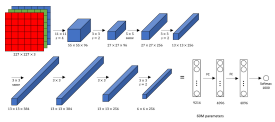
- Input: 3 channels (RGB)
- Conv filters: 96 filters
  → Output: **96 channels**

This is how AlexNet goes from:

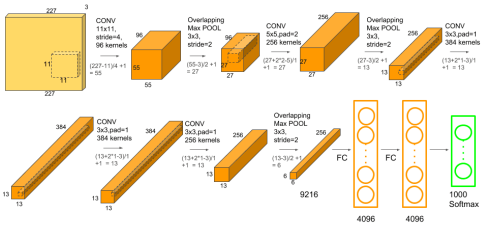
RGB (3 channels) → 96 → 256 → 384 → 256 → …

AlexNet example from lecture

Here's the relevant picture to anchor it:



Mr. Chat helped me connect the AlexNet diagram from lecture to the underlying tensor shapes. Sahai drew the block diagram (3→96→256→384→256 channels) and verbally mentioned that convolution layers change channels while pooling layers only change spatial resolution. However, in the moment it was easy to lose track of which operation was doing what. The AI's answer restates the rule in a very crisp way: pooling keeps the number of channels fixed, convolution changes them, and the number of filters equals the number of output channels. That is completely consistent with the lecture and with standard CNN practice. It also avoids a common hallucination: some models mistakenly say pooling can change channels, but here it doesn't make that error. As a learning tool, this section is a good example of "AI as structured note-taking": it reorganizes the verbal explanation from lecture into a set of simple rules I can refer back to when I'm working through homework or discussion problems on CNN architectures.



Let’s break down the early layers:

INPUT

224 × 224 × **3** (RGB)

Conv1 (5×5 stride 2, 96 filters)

→ 55 × 55 × **96**

Convolution increased channels → 3 → 96.

MaxPool1

(pool 3×3 stride 2)

→ 27 × 27 × **96**

Pooling kept channels the same.

Conv2 (256 filters)

→ 27 × 27 × **256**

Channels increased → 96 → 256.

MaxPool2

→ 13 × 13 × **256**

Again, channels unchanged.

Final Summary

✓ Convolutions change channels

- Number of filters = number of output channels
- This is how depth increases in CNNs
- AlexNet grows channels: 3 → 96 → 256 → 384 → 256

✗ Pooling never changes channels

- It only reduces spatial size
- e.g., 27×27×256 → 13×13×256

If you want, I can also explain:

- Why “increasing channels” increases expressive power

- Why deeper CNNs keep shrinking spatial dimension but grow channels
- Why early layers have few channels and later layers have many

Just tell me!

You said:

oh wait when we have 96 channels already to get to 256 we need 256 filters not 256-96?

ChatGPT said:

This section corrects a misunderstanding I had about how filters and channels relate, and it does so in a way that builds directly on Sahai's "perspective 2" view of convolution. The AI clarifies that each filter is a K×K×C_in *block* and that you have one such block per output channel, rather than one filter per input channel. That lines up with the lecture's second perspective where each spatial position in the kernel has a whole C_in×C_out matrix attached to it. I appreciate that the AI explicitly spells out the filter bank shape (e.g., 5×5×96×256) because that's exactly the kind of detail I tend to gloss over if it's only hand-waved in class. There is a bit of extra commentary about "depthwise vs pointwise convolutions" that goes beyond the 182 lecture material, but it stays in a parenthetical, so it doesn't derail the main point. Overall, this answer feels like a very faithful elaboration of what Sahai was getting at when he contrasted the two perspectives on convolution early in the lecture.

Exactly —**you need 256 filters**, NOT (256–96).
This is a *super* common confusion, so let's clear it up perfectly.

⭐ Key Rule

✓ The number of output channels = number of filters

✗ NOT "filters minus old channels"

✗ NOT "one filter per old channel"

✓ Each filter *uses all input channels*.

🧱 Why?

A convolutional **filter** is not a 2D matrix — it is a **3D block**:

filter shape=(K×K×Cin)filter shape=(K×K×Cin)

So when you have:

- Input with **96 channels**
- And you want output with **256 channels**

You make **256 filters**, each of size:

5×5×965×5×96

Because:

✓ Every filter "looks across" all 96 input channels

✓ Each filter produces one output channel

✓ So 256 filters → 256 output channels

Concrete example from AlexNet

Input to Conv2:

27×27×9627×27×96

Conv2 uses 5×5 filters and wants 256 output channels.

So filter bank shape:

5×5×96×2565×5×96×256

That last "256" means:

> "We have 256 different filters,
> each producing one output channel."

And that gives output:

27×27×25627×27×256

WHY it is not (256 – 96)

Because **filters do not exist "per input channel."**

Instead:

- Input channels = depth of each filter
- Number of filters = number of output channels

It's like this diagram:

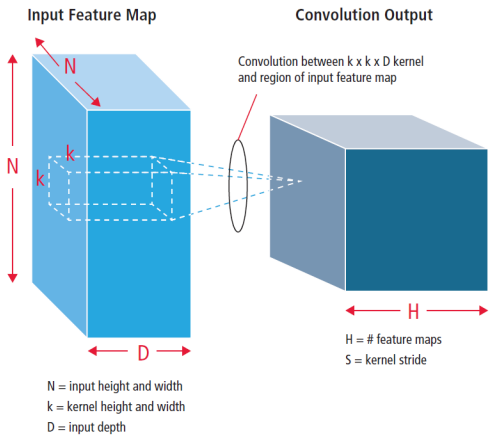
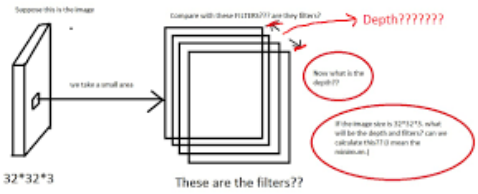


Figure 6: Pictorial representation of convolution process [9]



Each filter:

- spans *all* input channels
- creates *one* output channel

TL;DR

If you want to go:

96 channels → 256 channels

You need:

256 filters, each of size K×K×96256 filters, each of size K×K×96

Not 160 filters, not any subtraction.

If you want, I can also explain:

- How channels grow in CNNs
- Why increasing channels increases expressive power
- How filter banks are stored in memory
- How depthwise vs pointwise convolution relate to this

Just let me know!

You said:

someone was asking about order w and how we were saying that we update order 1 so we shouldnt loe up and muon and stuff i

didnt understand any of this please explain

ChatGPT said:

Got you — this part confused **a ton** of people in lecture, because he mixed:

- MUP scaling
- RMS norm layers
- what "order-1" means
- how repeated updates break assumptions
- why normalization layers are needed (speed bumps)

Let me explain the ENTIRE thing clearly and simply.

✅ FIRST: What does "order 1" mean?

In this class, **order 1** means:

✓ A value that is "normal sized", not exploding, not vanishing

✓ It stays within a constant scale, like 0.1, 1, 2, 5

✗ Not depending on the network width

✗ Not blowing up as layers or width get large

So when he says:

> "The updates should be order 1."

He means:

> **Updates should not explode or vanish. They should stay in a healthy range.**

✅ SECOND: What was MUP (Maximal Update Parametrization)?

MUP makes guarantees like:

- if the **inputs** to a layer have RMS norm ≈ 1
- and your initialization is Xavier/He (spectral norm ~ O(1))
- and the **updates** to the weights have the right scale (O(1))

then:

✓ Training will remain stable

✓ Gradients will not blow up

✓ Features will not blow up

✓ Updates won't depend on width

MUP gives a **theoretical recipe** to make updates "behave nicely".

BUT — here is the key problem he pointed out

This is where the AI starts to stretch beyond what was explicitly proved in lecture, and that's both a strength and a potential pitfall. On the positive side, the explanation of "order 1" as "not exploding or vanishing, and not depending on width" gave me a concrete mental model for how Sahai was using that phrase. The connection between MUP, RMS norms, and stability also matches the spirit of the lecture: he repeatedly emphasized that these scaling theories assume RMS norms stay in a healthy range and that normalization layers act like "speed bumps" to enforce that. However, the AI also adds extra details (like specific comments about FP8/FP16 overflow and spectral norms) that were not spelled out at that level of formality in class. I wouldn't call this a hallucination, since it's consistent with standard deep-learning lore, but it's definitely going beyond "what was said on the slides." As a learning tool, this is useful if you treat it as "enrichment," but it's important to remember that the exam will probably stick to the higher-level story the professor emphasized.

MUP **assumes** the RMS norm of the activations stays ~1.

But in real training:

You do millions of updates.

Even if every update is order 1,
small drifts accumulate:

- the RMS norm may become 1.3
- then 2
- then 5
- then 10

Now the assumptions break.

And then:

gradients get huge

activations blow up

FP8/FP16 overflows

nan nan nan

This is when training collapses.

THIRD: What normalization layers do

Normalization layers (RMSNorm, BatchNorm, LayerNorm) are:

→ “SPEED BUMPS”

that keep RMS norms from drifting away.

He literally said:

> “We want to enforce that the RMS stays 1.
> If the assumptions break, force them.”

So RMSNorm does:

$h \mapsto \frac{h}{\mathrm{RMS}(h)} h \mapsto \mathrm{RMS}(h) h$

This **explicitly forces RMS = 1 every layer**.

So now:

✓ MUP assumptions remain true across iterations

✓ The update scale stays healthy

✓ gradients don’t blow up

✓ training remains stable

✓ FP8/FP16 doesn’t overflow

Putting it all together

Here is a perfect timeline of what he was saying:

---

(1) MUP gives a theory:

If RMS(input)=1 and initialization is good and updates are O(1),
→ training stays well-behaved.

---

(2) But in practice:

After many updates, RMS(input) ≠ 1 anymore.
Even if each update is O(1),
many small O(1) steps accumulate → drift.

---

(3) This drift breaks the theory:

Gradients start scaling like √N or N,
Conv weight sharing adds more growth,
FP16/FP8 overflow → NAN.

---

(4) Solution:

Use **normalization layers** to force the RMS back to 1 before it grows.

> "It's duct tape.
> If you need a property and you can't guarantee it, enforce it."

FINAL SUMMARY (super clear)

"Order-1 updates" means:

Updates won't explode if everything else stays balanced.

But during long training:

Activations stop being balanced → RMS norms drift → training destabilizes.

RMSNorm/BathNorm/LayerNorm:

Reset the scale so things stay balanced.
They exist to make MUP's assumptions true *at every step*, not just at initialization.

If you want, I can draw a simple diagram showing:

- where RMSNorm goes
- how it resets the RMS
- how gradients flow through it

Just tell me!

You said:

wait so the theory is just wrong? cause it says it shoudnt blow up

ChatGPT said:

Great question — and**no, the theory is not wrong**.
The problem is **the assumptions of the theory stop being true** during long training.

Let me explain it the way the professor meant.

⭐ 1. What the theory ACTUALLY guarantees

MUP (and similar scaling theory) says:

> **"IF the input to every layer has RMS norm ≈ 1**
> **AND IF the weight matrices have spectral norm ≈ O(1)**
> **AND IF the updates are O(1)**
> **THEN the network will not blow up."**

This is a **conditional guarantee**.

It's not a universal guarantee.

It's like saying:

> "IF you keep your car aligned every month, the tires won't wear out."

The car isn't broken —
the *conditions* just need to keep being maintained.

⭐ 2. But during real training, the conditions stop being true

Here's what happens in practice:

1. At initialization, MUP conditions hold perfectly.
2. After 10k steps, still mostly fine.
3. After 200k steps, **activations drift** (RMS becomes 1.2, then 2, then 5…).
4. Once RMS ≠ 1 anymore → the theory's assumptions break.
5. Now the gradient scaling rules no longer work.
6. Things blow up.

So the theory wasn't wrong —
the *environment* it assumed drifted away.

⭐ 3. Why does drift happen even if updates are "order 1"?

This is the exact thing the professor was emphasizing:

Even an order-1 update, applied repeatedly, accumulates over time.

Imagine:

- Update size ≈ 0.01
- You apply it 200,000 times

You don't stay near the starting point.
Small constant steps → **big accumulated drift**.

So the RMS norm after many steps:

RMS≈1→1.1→1.5→3→10RMS≈1→1.1→1.5→3→10

Once it's 10, the MUP stability proofs no longer apply.

The two answers here together are a nice example of the AI helping me think about theory versus assumptions in a more mature way. In lecture, Sahai emphasized that MUP-style scaling results are conditional: they say "if RMS norms stay under control and updates are scaled correctly, then things won't blow up." But it was easy for me to mis-hear that as "this theory guarantees nothing will ever blow up in practice." The AI does a good job leaning into that distinction: it explicitly separates what the theory *assumes* (RMS ≈ 1) from what it *concludes* (updates and activations behave nicely) and then points out that real training can violate those assumptions over many steps. I also found it helpful that it defines an "update" concretely as $\Delta W = -\eta\, \partial L/\partial W$, instead of leaving it as a vague word. The risk here is that the car-alignment and Newtonian-mechanics analogies might feel a little informal or off-syllabus for a theory course, but as a learning tool they actually made the message stick for me much better than the original live explanation.

⭐ 4. So normalization layers FIX the broken assumptions

This was the professor's whole point:

> "If the assumptions break, FORCE them to hold."

RMSNorm/BatchNorm/LayerNorm:

- take whatever activations you have
- **renormalize them to RMS = 1 again**
- so the MUP assumptions remain true at every training step

This keeps training stable **indefinitely**, not just at initialization.

⭐ 5. So is the theory wrong?

NO.

The theory is correct **within its assumptions**.

The problem is:

❌ Real training violates those assumptions over time

✓ Normalization layers restore them

This is exactly like saying:

> "Newtonian mechanics is correct for slow speeds —
> but if you go near light speed, its assumptions fail."

Same vibe.

⭐ 6. The professor's real message

Let me restate it in his style:

- The MUP theory works **if RMS norms stay controlled.**
- But repeated training updates make RMS norms drift.
- So the theory *would* fall apart.
- To prevent that → normalize activations layer-by-layer.
- This is why RMSNorm/BatchNorm exist.
- They are "speed bumps" to enforce the scale assumptions of theory.

TL;DR (perfectly short)

The theory is not wrong.
Your network just stops satisfying its assumptions.
Normalization layers exist to keep the assumptions true.

If you want, I can also draw the "drifting RMS norm → crash → normalization resets it" picture as a simple diagram.

You said:

so the theory doesnt say that rms nrom will stay 1? it only says about updates? what even is an update

ChatGPT said:

Perfect — let's clear this up **cleanly** because this is the heart of the confusion.

The theory **does NOT** say:

> "RMS norm will stay 1 forever."

It never promised that.

The theory **assumes**:

> "If the RMS norm is 1,
> and if updates are scaled right,
> then things will behave well."

It's a **conditional** statement, not a guarantee about what happens over long training.

Now let's break everything down clearly:

⭐ 1. What is an update?

An **update** is simply:

> "How much you change the weights after one step of gradient descent."

Mathematically:

$W \leftarrow W - \eta \partial L \partial W W \leftarrow W - \eta \partial W \partial L$

The **update** is the term:

$\Delta W = -\eta \partial L \partial W \Delta W = -\eta \partial W \partial L$

So:

✓ "The update" = the amount the weights change each step

✓ Controlled by:

- the gradient
- the learning rate
- the optimizer (Adam, Muon, SGD, etc.)

When he says:

> "Updates must be order 1"

he means:

> "ΔW should be normal size, not exploding or vanishing."

⭐ 2. What does the MUP theory actually say?

Something like:

> "If inputs to a layer have RMS norm ≈ 1,
> and if the initialization is scaled right,
> and if the update ΔW is scaled O(1),
> then the *activation scales* and *gradient scales* will remain well-behaved."

It is NOT a claim that RMS norm stays 1.
It is an assumption that RMS norm ≈ 1 at each layer.

⭐ 3. So what breaks in practice?

I like how it gives me this checklist and then also a quote at the end to make it easier for me to relate to physics for example and why it breaks.

MUP theory subtly assumed:

- You update weights
- But activations don't drift too much
- So RMS norm remains “reasonably near 1”

But in practice:

✓ You do millions of small updates

✓ Small changes accumulate

✓ Activations drift

✓ RMS norms grow

✓ Now the assumptions break

✓ Theory stops applying

✓ Training becomes unstable

✓ You get NANs or divergence

This is NORMAL — it’s not a flaw in the theory.

It’s like saying:

> “My physics formula assumed no air resistance —
> but now I have air resistance so the formula doesn’t hold.”

The math wasn’t wrong — the *conditions changed*.

⭐ 4. Why RMS Norm / BatchNorm fix this

Normalization layers explicitly:

$h \mapsto \frac{h}{\text{RMS}(h)}$

meaning:

> **They RESET the RMS norm to 1 every forward pass.**

This ensures the MUP assumptions remain true at all times.

Without it, RMS norm drifts → assumptions break → training blows up.

⭐ 5. Final Clarification (SUPER IMPORTANT)

✗ The theory does NOT say RMS norm stays 1.

✓ The theory ASSUMES RMS norm stays 1.

These are completely different ideas.

This is why normalization layers are needed:

→ To enforce the assumption

→ So the theory continues to apply

→ So training stays stable

Normalization is **not replacing the theory** —
it is ensuring the *theory's conditions* hold during training.

TL;DR

- **Update** = how much weights change at each step.
- MUP theory assumes activations have RMS norm ~1.
- But training causes RMS norms to drift.
- Once RMS norm ≠ 1, assumptions break → instability.
- RMSNorm/BatchNorm force RMS norm back to 1 → keep theory valid.

If you want, I can give you a **visual timeline** of how RMS norm drifts with and without normalization — that often makes the entire thing crystal clear.

You said:

Actually, maybe 0:13 it's better to put it here. 0:27 Better. 0:39 I can see the students. 0:56 Okay. So hopefully this works. 1:01 Does this mic help you here or No, it does not at all. I don't think you can hear anything 1:07 clearly. Okay. Can you hear? Yeah, you can't hear this mic. Okay. Recording stopped. 1:16 Okay. I'll just come and repeat questions loudly to the computer. Recording in progress. 1:21 Okay. Okay. Let's try this. Okay. Thanks everyone. Hopefully, can 1:27 you hear me? Raise your hand if you can hear me. Okay, great. Thank you. Sorry for not being there in person. Uh try to 1:34 deliver the material properly uh remotely. So, we're going to continue our discussion of uh convolutional neur 1:40 neural nets. The story is as it was before. Uh you know, we want to do image 1:46 processing. We want to do appropriate word sharing using the convolutional architecture. But I realized there's something that I 1:52 didn't uh uh talk about uh clearly last time and I want to make sure it is clear 1:58 to everybody. So when we think about a k by kcon with c input channels and see 2:05 out output channels uh we have two different perspectives on it. Of course in terms of the computations both 2:11 perspectives are identical. Just how do we think about it? The number of parameters is also identical. you know 2:17 we have k^ squ cnc see out weights and see out biases again as before. So the 2:23 first perspective is the one that we are kind of describing. Uh let me make sure 2:29 everyone understands it. And the way that you think about it is you think about it as see out different K by K 2:38 cons each of these K by K cons having C and input channels and one output 2:43 channel. Um why? Because that's the picture uh that we see here. Uh hopefully everyone 2:50 can see this picture. So here we see a single output 2:56 with uh k by k here's three by uh 3x3 con with uh cn here visually three uh 3:04 input channels. So again everything corresponding to a particular location in the output comes from a filter acting 3:12 on that centered same location on the input where the filter has a different 3:18 uh 3x3 uh matrix of real valued weights associated with every channel. Those 3:24 things are all multiplied together and added up add together the bias. So in this picture we have the following 3:32 picture for one output channel just simply repeated see out times to give you see out different outputs. 3:41 This picture uh is hopefully clear. 3:46 Uh I want to contrast this with a second perspective a different perspective. Again the result is the same but it's a 3:52 different way of looking at it. And this uh other perspective says look 4:00 we're going to think about this uh as a single k by k con except instead of 4:07 having weight numbers we have weight matrices in every box and instead of 4:14 having a bias number a single bias we have a bias vector uh in every box 4:20 not every box but sort of for for the conf. So here's a 3x3 example kind of worked out together with the kind of 4:27 described with the math. So before we drew this and you thought of each of these W's, the one corresponding to the 4:33 pixel itself, the one corresponding to north and so on as all being scalar values, real values. Now we think of 4:42 each of these for example let's look at WN. Uh they all all the W's have the same shape. They are a CN by C out 4:50 matri. So CN inputs. So CN in this case we're using a vector convention of of 4:55 columns. So we have C in columns and C out rows and the bias for the entire 5:02 conf is a vector with the seout uh size dimension. 5:09 So if you think about it in terms of a formula what you have is the output assuming nonlinearity here is just the 5:15 bias. So remember the output is for per pixel everything is weight shared across pixels. So a bias vector plus a sum a 5:25 sum over i where i is this whatever this neighborhood structure is. So for a k by k uh there's k squ terms in the sum 3x3 5:34 there's nine of the corresponding weight matrix wi acting on the corresponding 5:41 input vector from the uh image at the previous level. So this perspective says 5:46 that what happens at cons in connets is that every you always maintain an image 5:52 and the entries of every pixel are vectors. There were three vectors in the case of RGB and they are whatever 6:00 generic vector size we want for deeper in the network. Okay, I

want to pause 6:05 here and just take any questions to make sure people understand these two are equivalent to each other. They both 6:11 represent a learnable linear uh so aine transformation from uh one image to 6:18 another. But uh I want to make sure everyone understand these two perspectives. 6:23 Any questions on this? Yeah. 6:33 I in the summation refer to the number of channels. Uh great. The question was what does the 6:39 I refer to in the summation? The I is not referring to the number of channels. Let me write it out. 6:49 So in this particular sum I this is ranging over 7:01 the A by K 7:06 positions. in the con filter. 7:15 Okay, so it's not summing up uh over channels, it's summing up over spatial 7:21 positions here where remember the spatial position, there's one weight that 7:26 corresponds to me, the center of the filter, and then there's its local neighborhoods. 7:32 So the H out for this pixel is going to be the sum over I the neighborhoods the 7:39 3x3 neighborhoods of the weights acting on the corresponding pixel in the input. 7:46 So the corresponding pixel here for me is the same exact pixel position. The 7:52 one for north, so there's a north in here, is the pixel that's one above the 7:57 current pixel. For south, the pixel that's one below. For east, one to the 8:04 right. And for uh sorry, for west, you know, one to the left. Does that make it 8:11 clear? Does that answer the question? Yeah. 8:16 Yeah. Let's try this just to see if it works. If it doesn't 8:24 um where in the formula do we account for the number of channels? 8:30 Uh I heard som, we can just do exactly what we did during training 31:16 because layer doesn't look at batches. It doesn't it doesn't do any combination 31:22 of information across the across the batch. It just looks at everything one at a time. So at inference time or 31:28 testing time we have only one thing at to work with. So there's no issue of layer but there is an issue for batch. 31:44 You don't have a batch. 31:56 So what do we do for the case when we don't have a batch? 32:02 So what we need 32:08 is M and S. We can't do this averaging to get the 32:15 M&S. So we still need it to apply the formula. So if we had MNS, we could 32:21 apply this formula for the output of batch norm. 32:26 Okay, so we need to get an M&S. So the question is how do we get an M&S? 32:39 So at this point I'll let you guys think for a little bit. Hopefully everyone understands the 32:44 question. So now take a minute, talk to your neighbors and say given that you 32:50 have the uh the setup that you have, how do you get these n and these s's to run 32:56 bashnorm at testing time? What what do we have to do to get them? Now when I 33:02 say to get them, you have many choices. You could probably do something purely at testing time. You could also do 33:07 something different uh during training to get yourself M&S's. Lots of possibilities, but just talk to your neighbors for a minute and talk about 33:13 and figure out what you might do. 33:40 What is the distribution of their 34:05 That's what you Oh, you're testing. Oh, I see. 34:24 Okay, I'll try to call everyone back together. Okay. 34:30 So, hopefully you've had a chance to think about it. Does anyone in the room 34:35 want to volunteer a a suggestion of what we could do? 34:46 Um, we were thinking we could keep like a running average of the mean and standard deviation on like the last 34:52 epoch of the training and use that during inference. Okay, great. That's a great question, a 34:57 great approach. So, let me write that out. One choice. 35:05 Just get an average 35:13 for M and S 35:19 from the last epoch of training. 35:26 Uh that's a totally valid uh thing to do. Um you can also just run one epic 35:32 with no updates for this purpose. Okay. Alternative 35:48 training 35:55 with no gradient updates. 36:04 Okay. So why do I say that? Because this is ensuring that the M ands that you 36:10 compute as this average is good for whatever the current parameters are of the network. The parameters aren't 36:15 changing at all. Is everyone clear with clear with this about this approach? Any 36:20 questions on this approach? It's a way of getting the M&S that you don't have. Okay. Does anyone have any other ideas 36:27 for what one can do? This is the standard thing to do by the way, but do you have any other ideas what you could do? um very similar, but you could run 36:34 uh like a running exponential average over the last several epochs um and use 36:40 those values for M&S. Okay, so thou learn to to add up numbers and you have 10 items with uh a 54:13 pres preservation probability of 0.9 and it learns like oh I want to keep adding to nine so I'm going to make everything 54:18 one right but all of a sudden when you go to testing time and you keep everything activated since your network 54:25 has like learned to make each entry one all of a sudden when you keep everything you hit 10. So all of a sudden sort of 54:32 the scale you're getting to exceeds because you simply have more of your values at your disposal. 54:40 Yep. Excellent answer. It messes up scale. 54:48 So remember one of the most fundamental things we learned about neural nets that drove our discussion of optimization and 54:54 thinking about RMS norm and so on is that these neural nets like to have their activations and pre-activations in 55:00 a particular range around zero you know in some near vicinity. If suddenly you 55:07 are not drop 1:15:28 using a super highway. So the gradient if there was only uh there only this 1:15:33 connections and the gradient's going this way to everybody but what would help this layer that 1:15:41 wouldn't have a path to go to everybody except through this backward long chain. So that's what we thought about adding 1:15:46 paths from everything to the everything else. That's too many paths. This is kind of in between. So the gradient here 1:15:54 goes back to everybody, but the gradient from that's coming through here is also now read this lecture. in the very beginging when he is talking about perpective 2 the i in the sum what is that? only the neightbohood of the filter and if so what dos that mean

ChatGPT said:

Great — you're asking exactly the right thing.
Let's zoom into the **beginning of the lecture** where he describes:

> **Perspective 2** of a convolution
> where each weight is a *matrix* and the formula is:

h_{\text{out}}(i,j) = b ;+; \sum_{i \in \text{filter positions}} W_i , x_{i}
]

You are asking:

> **What is the i in that summation?**
> **Is it summing over channels? Or pixels? Or something else?**